Prototype Project

ME 2900

Instructor: Kai Vogeler

Date of Submission: 3/4/2026

Alex Hertzfeld

I. Introduction

For this project I created an oscillating fan that is temperature controlled and has a light attached that is controlled by the light level in the room. The fan speed can also be adjusted by using a button. The goal of this prototype was to create a system that automatically regulates airflow to improve room comfort, because I often struggle with maintaining a comfortable overall temperature in my room. Ultimately, I wanted to create a contraption that could detect a certain temperature and react by adjusting airflow accordingly, in which, reducing manual work. Since this devise would be placed on my desk, I added a LED Light to serve as a function desk lamp.

In order for this project to solve the goal it should be able to pass some basic requirements. These requirements include being able to read room temperature accurately, turning on when a set temperature is detected, oscillating around 90 degrees, must be safe to operate, light turning on when dark, and fan speed can be manually controlled. The reason it must be able to read room temperature accurately and turn on when a set temperature is detected is to circulate airflow for when the room gets hot. If it doesn't turn on or isn't accurate with the reading, it won't be working for it intended use. The oscillating and manual fan speed is to get better airflow in the room and cool it down quicker. Having the device be safe to use is a must as it is intended to be close to where people will be frequently. The light should turn on only when it is dark to use as a desk lamp but yet save energy. Some Non-Functional requirements include it being simple to operate and be compact but yet also hide any wires or electronics.

With further iterations, my goal would be to make the design more compact, durable and polished looking. This could be achieved by organizing where all of the components are and minimizing unused space on the breadboard, along with 3d printing parts to allow for a more exact fitting and strength in the material. Overall, the final product would resemble a compact looking desk fan with a clean and simple appearance. In order to be competitive in the market it also must be cheap to make in order to sell for cheap as well.

II. Idea Development Process

After deciding what I wanted to make, I started by figuring out what were the main components that I would need in order to make this work. So, I started small and worked to the big picture. To begin with, I focused on the fan. I figured it would be best to use the DC motor as it spins fast and can be controlled speed wise with

pulse width modulation. Next, I looked at how to make it oscillate. For the oscillation, I picked the servo motor as it turns on an axis and can also be set to different positions. I then decided to use the I2C sensor as the temperature sensor as I had used it in an earlier in a lecture assignment. After, I knew that we had used a system in a earlier lecture assignment that acted as a nightlight, I figured that this would be perfect to use for the desk lamp aspect. Finaly, I settled on the simple buzzer to signal the light had turned on as its task is very basic and isn't needed for much.

To test my brainstorming above I started in the same order I brainstormed in. I first made sure I was able to code and control the motor doing pulse width modulation. After a successful test, I added the button that when pressed would change the fan speed. This turned out to be harder than expected as the button kept bouncing and changing values. To solve this problem, I used the button debouncing code from lecture and arranged it to fit in my code. I then moved to the I2C sensor to make sure that it only turned on when it was hot in the room. Followed was the servo motor where I got the motor to turn as wanted and then implemented into the temperature sensitive part of the code. The final bit of testing was including the nightlight lecture code and hardware along with the active buzzer and pitches. These where relatively quick as we had already done something similar in lecture activities

This was not a simple and straight forward project. I went through many different designs and components before my final. My original idea to solve the problem was to make a contraption that would automatically close and open my vents in my room. The reason I didn't do this is because finding a way to grip and move the vents was a struggle and required more power and torque than I was able to generate. I also thought about using a thermistor instead of the I2C sensor in order to detect the temperature. The reason I decided against this is because getting an accurate reading and turning it into Fahrenheit proved to be difficult compared to the I2C sensor that I had already used previously. I had also wanted to use and LCD display to show the current temperature it was reading. Due to the fact that the DC motor takes so much of the Arduino's energy when running, it messed with the LCD display causing it to look weird. Because the LCD was not important to the functionality of the prototype I decided to remove it.

My programming logic also needed to be changed and edited to account for the DC motor. My original logic was that it would read all the sensors while the fan was still running. However, with the DC motor taking all the energy from the system when

it was running, I had to temporarily pause the motor and allow all the sensors to read and output the changes before then starting the fan again. The other way I considered solving was to increase the power supplied to the system. This didn't work however as the Arduino can still only handle and output its set voltage. I also considered running the motor on its own power supply but decided against it as it added some unnecessary extra components.

III. Discussion

The finished prototype is a temperature-controlled oscillating desk fan with a light that reacts to the light levels. When powered on, the system will continuously read the room temperature using the I2C temperature sensor. If the detected temperature is above the programmed threshold (75* F) the fan begins to spin and oscillate. The fan speed and be controlled with pulse width modulation, allowing for adjustable airflow levels.

The user is able to interact with this device using a push button. Each press of the button cycles between preset fan speed levels. To ensure reliable input, button debouncing code and logic is implemented in the program to prevent unwanted extra readings from a single press.

To distribute airflow better, a servo motor is used to oscillate the fan. The servo between set angles of 0 and 90 degrees, creating a sweeping rotation that spreads air across a wider area of the room. This oscillating activity is active when the temperature threshold is reached and the fan is active.

In addition to the temperature control, the prototype includes a photoresistor that monitors the light levels in the room. When the room becomes dark, an LED activates and can function as a desk lamp. An active buzzer provides an audible noise when the light tuns on, to alert the user of the change.

The system runs of a continuous flow of sensor inputs, coding logic and actuator outputs that are all controlled by the Arduino. The inputs in the prototype include the I2C sensor, the photoresistor, and the button, The outputs include the DC motor, servo motor, the LED, and the active buzzer.

In the setup portion of the code, all input and output pins are initializes, serial communication is begun and all the libraries are included. Also included are the user-defined functions that are used to organize the code. These functions include the button debouncing code, temperature reading code, nightlight code and active reading the button inputs code.

The main programming logic can be found in the void loop of the code. The flow of logic and information goes as follows:

Step 1: The system reads the current temperature from the I2C sensor

Step 2: Compare temperature to the set temperature threshold of 75* F.

Step 3: If temperature exceeds threshold, a signal is sent to the DC motor and to run at the lowest PWM level. If it isn't met it returns to step 1.

Step 4: When fan is active, commands are sent to the servo motor to move to set positions, oscillating the fan.

Simultaneously during all 4 steps, the program monitors the photoresistor output and if it falls below the set threshold, the Led is turn on and the buzzer is activated to provide the noise indication.

User input from the button is used by the button debugging code logic where the program compares the current and previous button states to detect a valid press. Each press increases the counter that then is used the cycle through the preset PWM values that correspond to the different fan speeds.

Due to the high current draw of the DC motor, the system was changed in which to temporary pause the motor to allow for the sensor reading to be taken. This ensures that appropriate voltage was used to get correct readings from the different sensors.

IV. Conclusion

On the more successful aspects of my prototype having the DC motor rotate and be able to read the outputs of the different sensors. Coordinating the DC and all the sensors together required meticulous programming and logistical structuring. I was particularly proud of how the system was able to read and respond the environmental changes without requiring constant user input. Seeing all of the sensor accurately show the environment around them in a short time to allow for it to seem like the fan didn't even turn off, showed the system working in perfect harmony.

As already mentioned, the biggest challenge I encountered was the power interference that was caused by the DC motor. When the motor was running, it drew a large amount of current from the Arduino, this lead to unstable senor reading and issues with other components. After troubleshooting, I found I needed to modify the program logic to temporarily pause the DC motor to allow for sensor to take

readings. I had also considered adding an alternative power source or finding a way to run the DC motor on its own separate power. This challenged helped me understand power distribution limitations in small systems and the importance of managing the currents when working with the different motors.

Given more time and resources, I would start by improving and redesigning the power system and overall structure. I would power the DC motor using independent power supply to prevent any effects that it causes the issues with the sensors. This will remove the need to temporarily pause the motor and instead could constantly be reading it. Additionally, I would replace the cardboard outside with a 3D printed structure to allow for more compact, durable and a better looking appearance. These improvements would allow for the prototype to function better and also look like a finished product.

Appendix A: Pictures of Prototype

Image 1: Full completed prototype

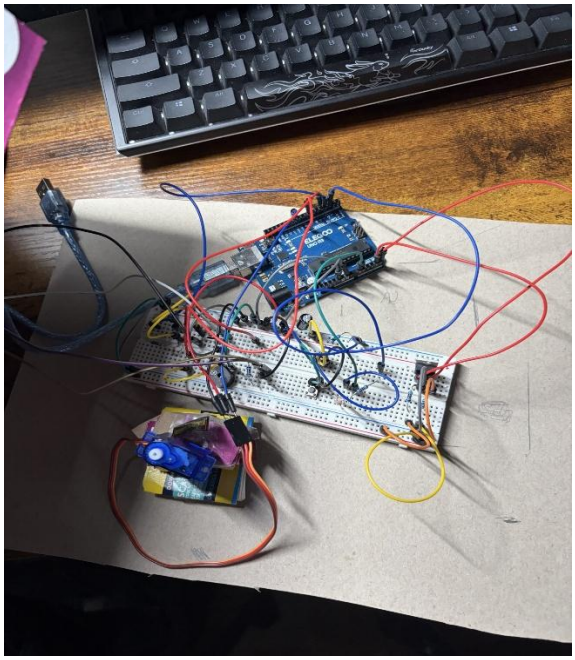

Image 2: Final circuitry

Image 3: Final tower fan

Appendix B: Circuit Schematics

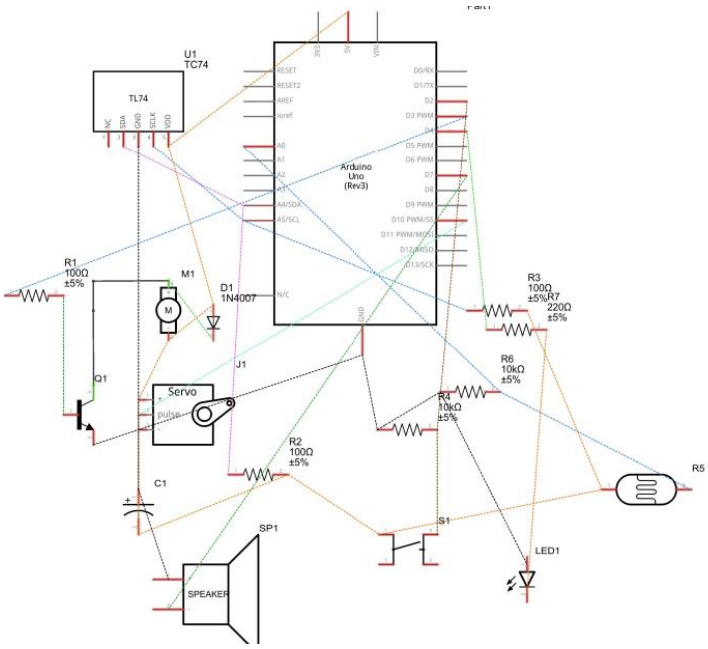

Figure 1: Final circuit schematic

Appendix C: Arduino Sketches/Libraries

Full prototype code:

```
// Importing Libraries

#include <Wire.h> //For I2C

#include <Servo.h> //For Servo

#include "pitches.h" //Header file with pitch definitions


//Initalizing Variables

Servo myServo;

const int motor_pin = 3;

int temp_address = 72;

int f;   // Variable to store Fahrenheit temperature

int pressCount = 0; //buton pressed amount

const int BUTTON=2;

int Servopos =0; //Servo position

boolean lastButton = LOW;    // Variable containing the previous

boolean currentButton = LOW; // Variable containing the current

boolean ledOn = false;      // The present state of the LED (on/off)

const int SPEAKER=7; //Speaker Pin

const int WLED=4;       // White LED Anode on pin 4

const int LIGHT=0;      // Light Sensor on Analog Pin 0

const int MIN_LIGHT=200; // Minimum Expected light value

const int MAX_LIGHT=900; // Maximm Expected Light value

int val = 0;           // Variable to hold the analog reading


// Reading Tempature output user-fucntion
```

```
void tempRead()
{
 analogWrite(motor_pin,0); //stopping motor
 delay(300); //dealy 0.3s

 Wire.beginTransmission(temp_address); //gathering temp
  Wire.write(0);
  Wire.endTransmission();

  Wire.requestFrom(temp_address, 1);

  int c = Wire.read();       // Read Celsius
  f = round(c * 9.0 / 5.0 + 32.0);  // Convert to Fahrenheit
delay(50);

}

//Button debounce user-function
boolean debounce(boolean last)
{
 boolean current = digitalRead(BUTTON);   // Read the button state
 if (last != current)                  // if it's different...
 {
  delay(5);                          //Wait 5ms
  current = digitalRead(BUTTON);        //Read it again
 }
```

```
 return current;                    //Return the current value
}

//Buttin being pressed function
void BPC()
{
currentButton = debounce(lastButton);                    //Read debounced state

 if (lastButton == LOW && currentButton == HIGH) //if it was pressed...
 {
  pressCount++;
  ledOn = !ledOn;                                  //Toggle the LED value
 }
 lastButton = currentButton;                              //Reset button value
}

//Reading button while DC motor on user-function
void waitAndReadButton(unsigned long duration) {
  unsigned long startTime = millis();
  while (millis() - startTime < duration) //While loop that runs only for durration of fan on
  {
    BPC(); // Keep checking the button during the wait
  }
}

//Photoresistor and LED turn on user-function
```

```
void nightLight()
{
val = analogRead(LIGHT);                    // Read the light sensor
 val = map(val, MIN_LIGHT, MAX_LIGHT, 255, 0); // Map the light reading
 val = constrain(val, 0, 255);              // Constrain light value
 if (val >= 100) // If dark
 {
  digitalWrite(WLED,HIGH); //turn on LED
  tone(SPEAKER,NOTE_A4,250); //Playing noise on speaker
 }
 else
{
  digitalWrite(WLED,LOW); //turning LED off
}
}

void setup()
{
  Wire.begin(); //Startign Wire for I2C
  Serial.begin(9600);
  pinMode(motor_pin, OUTPUT); //Motor pin as output
   pinMode (BUTTON, INPUT);    // Set button as input
    myServo.attach(10);
    pinMode(WLED, OUTPUT);  // Set White LED pin as output
```

```
}

void loop()
{
  nightLight(); //Getting light value
tempRead();; //getting temp value

Serial.println(f);
if (f >=75) //if temp threshold is met
{

BPC(); //read button



if (Servopos >= 91) //Move serovo back to start
{
  Servopos = 0;
}
if (Servopos <= 90) //Oscillating the Servo
{
    myServo.write(Servopos);   // Move to the next degree
    Servopos = Servopos + 15; //Add the other degree
}

if (pressCount ==0) //Defult fan speed
{
```

```
  analogWrite(motor_pin,127); //turn fan on at 50%
  BPC(); //read button
  waitAndReadButton(5000); //Delay 5s

}
if (pressCount ==1) //Next PWM level
{
  analogWrite(motor_pin,191); //turn fan on at 75%
  BPC();//read button
  waitAndReadButton(5000);//Delay 5s

}

if (pressCount ==2) //Next PWM level
{
  analogWrite(motor_pin,254); //turn fan on at 100%
  BPC();//read button
  waitAndReadButton(5000);//Delay 5s

}

if (pressCount == 3) //Resetting button count to go back to defult speed
{
pressCount = 0;
}
}
```

}

Refrences:

I2C wire: Inter-Integrated Circuit (I2C) Protocol | Arduino Documentation

Servo Control: Basic servo control | Arduino Documentation

Chatgpt: Used to help with user function of “void waitAndReadButton”. Asked for example code to work off of.

Speaker: Tone on Multiple Speakers | Arduino Documentation

Appendix D: 3D Rendering

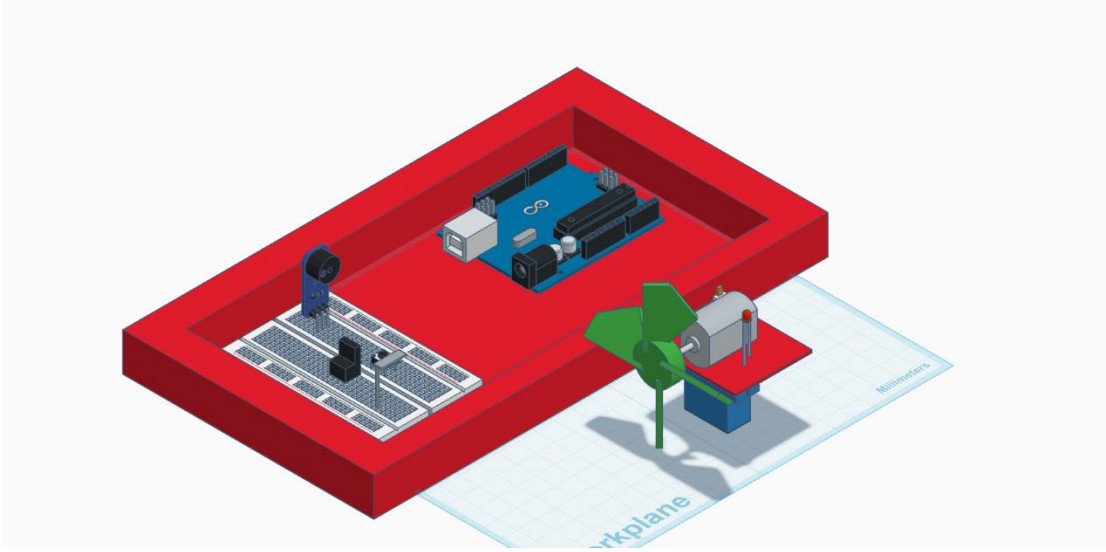

Figure 2: Full orthographic view of prototype

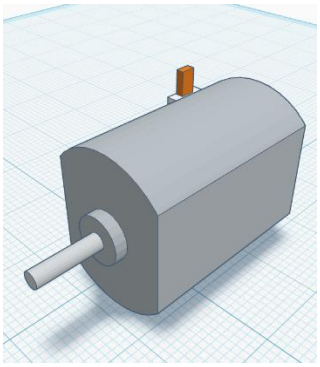

Figure 3: DC motor part

Appendix D: Video

Link to video: ME2900 Prototype Project